

Machine Learning Notes

Chapter 3: Generalization and Regularization

Vitor N. Cabral de Oliveira

vnco@cin.ufpe.br

August 18, 2026

In Chapters 1 and 2, we built an engine capable of mapping inputs to outputs and optimizing its weights via gradient descent. However, a highly expressive model trained without constraints will simply memorize the training data, failing catastrophically on unseen examples. In this chapter, we address the core challenge of machine learning: *Generalization*. We explore the bias-variance tradeoff, the modern phenomenon of double descent, and the mathematical foundations of regularization techniques like L2 Weight Decay, Dropout, and Batch Normalization.

Contents

<i>Optimization vs. Generalization</i>	3
<i>The Bias-Variance Decomposition</i>	3
<i>The Modern Reality: Double Descent</i>	4
<i>Regularization Techniques</i>	5
<i>L2 Regularization (Weight Decay)</i>	5
<i>Dropout</i>	5
<i>Batch Normalization (BN)</i>	6
<i>Early Stopping</i>	8
<i>A Summary of Regularization Techniques</i>	9
<i>Exercises</i>	9

Optimization vs. Generalization

In machine learning, there is a strict distinction between the training set and the test set. The model's ability to perform well on data it has never seen before is called **generalization**.

Recall our goal: we want to minimize the *expected risk* (the true error over all possible data drawn from the distribution). Since we don't have access to the true distribution, we minimize the *empirical risk* (the error over our training dataset $\mathcal{D}$).

If our model has millions of parameters, it can drive the training loss to zero by memorizing the exact input-output pairs. But memorization is not learning. When presented with a new input, a memorizing model has no underlying understanding of the data's structure and will output garbage.

The Bias-Variance Decomposition

To understand generalization mathematically, we look at the expected generalization error of a model for a target y and prediction $\hat{y}$. For squared error loss, the expected error can be decomposed into three parts:

$$\mathbb{E}[(y - \hat{y})^2] = \underbrace{(\mathbb{E}[\hat{y}] - y)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\hat{y} - \mathbb{E}[\hat{y}])^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible Error}}$$

- **Bias:** The error from erroneous assumptions in the model. A low-capacity model (e.g., a linear model trying to fit a sinusoidal wave) has high bias. This is *underfitting*.
- **Variance:** The error from sensitivity to small fluctuations in the training set. A high-capacity model (e.g., a deep neural network) can fit any noise in the data, meaning its predictions will vary wildly if trained on different subsets of data. This is *overfitting*.
- **Irreducible Error:** The noise inherent to the data generation process. No model can reduce this.

The classical goal of machine learning was to find the "sweet spot" in Figure 1—the model capacity where the sum of bias and variance is minimized.

To see the tradeoff concretely, consider fitting polynomials to a handful of noisy points sampled from a smooth curve. The degree of the polynomial plays the role of model capacity.

Example 0.1 (Bias, Variance, and Fit) The three panels of Figure 2 tell a quantitative story. Suppose the true function is $y = x^{1.2} + \text{noise}$. Evaluated on the same ten points:

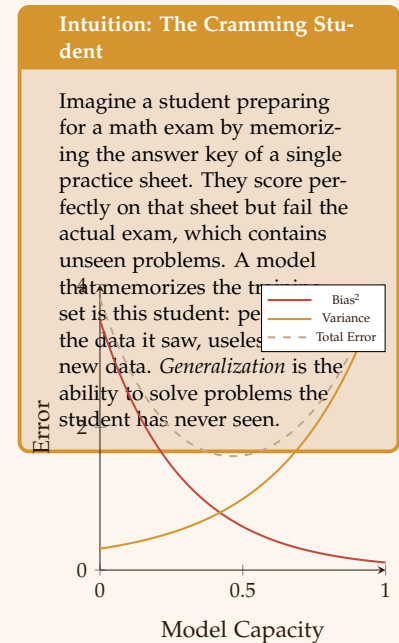


Figure 1: The classical Bias-Variance Tradeoff. Total error has a minimum at an optimal intermediate capacity.

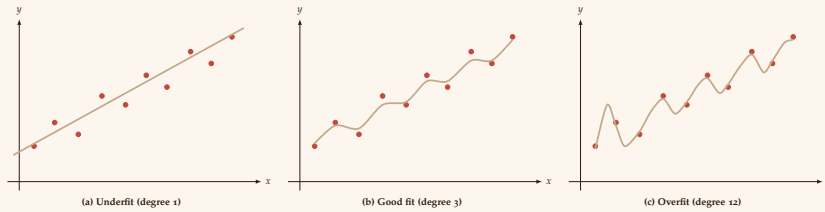


Figure 2: Polynomial fits to the same noisy data. (a) A degree-1 line cannot capture the curvature—high bias, low variance. (b) A degree-3 polynomial tracks the underlying trend—low bias, moderate variance. (c) A degree-12 polynomial wiggles through every point, fitting the noise itself—low bias but very high variance: it will perform poorly on new data.

1. **Underfit:** The straight line misses the curvature systematically. Averaged over many training sets, its predictions land near a consistently wrong value—large Bias^2 , small Variance.
2. **Good fit:** The cubic follows the trend. Its average prediction is close to the truth (small Bias^2) and does not change much across training sets (moderate Variance).
3. **Overfit:** The high-degree polynomial reproduces the training noise almost exactly. On a different training set, the wild wiggles would move dramatically—small Bias^2 but large Variance. The test error explodes even though the training error is ≈ 0 .

The Modern Reality: Double Descent

Classical statistics suggests that as model capacity increases past the sweet spot, the test error will rise indefinitely (overfitting). However, modern Deep Learning defies this dogma. If we make the model *massively* over-parameterized (far more parameters than data points), the test error drops again and often reaches its lowest point!

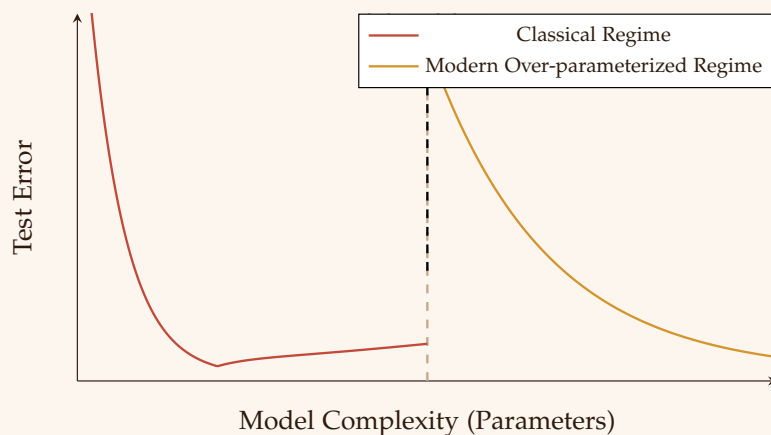


Figure 3: The Double Descent phenomenon. Past the interpolation threshold (where train error becomes zero), increasing model complexity actually decreases test error.

This phenomenon is known as **Double Descent** (Fig. 3). Neural networks don't just memorize; they search for the *simplest* function that interpolates the data. This modern finding explains why we can

train LLMs with billions of parameters without catastrophic overfitting. However, explicit regularization is still required to stabilize training and guide the model toward these well-behaved solutions.

Regularization Techniques

Regularization encompasses any technique designed to reduce generalization error, often at the cost of increased training error. We constrain the hypothesis space.

L2 Regularization (Weight Decay)

The simplest form of regularization adds a penalty term to the loss function that discourages large weights. For a loss $\mathcal{L}_0$ and weight vector $\mathbf{w}$, the regularized loss is:

$$\mathcal{L} = \mathcal{L}_0 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

When we compute the gradient, the L2 term adds $\lambda\mathbf{w}$ to the update rule. This causes the weights to "decay" towards zero at every step:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta(\nabla\mathcal{L}_0 + \lambda\mathbf{w}) = \mathbf{w}(1 - \eta\lambda) - \eta\nabla\mathcal{L}_0$$

Geometrically, L2 constraint creates a hypersphere in weight space. The optimizer is forced to find a solution that lies inside this hypersphere, preferring distributed, small weights over a few dominant ones.

Example 0.2 (Weight Decay in Action) Consider a single weight with $\eta = 0.1$, $\lambda = 0.5$, and suppose the data gradient is zero ($\nabla\mathcal{L}_0 = 0$)—the model has already fit the training set. The update rule becomes

$$w \leftarrow w(1 - \eta\lambda) = w \times (1 - 0.05) = 0.95w.$$

Starting from $w = 2.0$, the weight follows $2.0 \rightarrow 1.9 \rightarrow 1.805 \rightarrow 1.715 \rightarrow \dots$, shrinking by 5% each step. Even without any data pressure, the weight inexorably decays toward zero. The decay rate is controlled entirely by the product $\eta\lambda$: the same update can be implemented by multiplying weights by $(1 - \eta\lambda)$ after every step—which is why the technique is also called weight decay. When the data gradient is nonzero, the decay and the data term compete, and the weight settles where the two balance.

Dropout

While L2 operates on the loss function, Dropout operates directly on the network architecture. During training, a random fraction p of neurons in a layer is set to zero (dropped) during the forward pass.

Bayesian View of L2

From a probabilistic perspective, L2 regularization is equivalent to Maximum A Posteriori (MAP) estimation assuming a Gaussian prior over the weights. We are telling the model: "Assume most weights should be close to zero unless the data strongly suggests otherwise."

Listing 1: Dropout implementation in mini_nn/nn_utils.py.

```

1 import numpy as np
2
3 class Dropout:
4     def __init__(self, p=0.5):
5         self.p = p # Probability of dropping a neuron
6
7     def __call__(self, x, training=True):
8         if not training:
9             return x # During inference, use all neurons
10
11         # Generate binary mask. Neurons are kept with
12         # probability (1-p)
13         mask = (np.random.rand(*x.shape) > self.p) / (1.0 - self.p)
14         return x * mask

```

Dropout prevents *co-adaptation*. Without dropout, neurons might learn to rely heavily on the output of a specific upstream neuron. If that upstream neuron fails, the downstream neuron fails. By randomly dropping neurons, the network is forced to learn redundant representations—multiple paths must encode the same information.

During inference (testing), no neurons are dropped. However, because only $(1 - p)$ neurons were active during training, we must scale the activations during inference by $(1 - p)$ to maintain the expected output magnitude (or, as in the code above, scale by $1/(1 - p)$ during training so inference needs no scaling).

Batch Normalization (BN)

Deep networks suffer from *internal covariate shift*: as weights update, the distribution of inputs to deeper layers changes continuously, forcing those layers to constantly adapt. Batch Normalization stabilizes this by explicitly normalizing the inputs of each layer.

For a mini-batch $\mathcal{B} = \{x_1, \dots, x_m\}$, BN calculates the mean $\mu_{\mathcal{B}}$ and variance $\sigma_{\mathcal{B}}^2$:

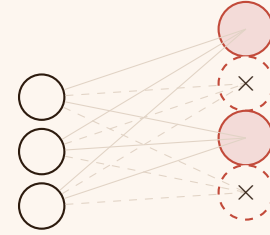
$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^m x_i, \quad \sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2$$

It then normalizes the inputs:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

Finally, it applies a learnable scale γ and shift β :

$$y_i = \gamma \hat{x}_i + \beta$$



Dropout Layer

Figure 4: Dropout during training. Active paths are solid; dropped paths are dashed.

The transformation is easiest to visualize as a three-stage pipeline applied elementwise to every activation:

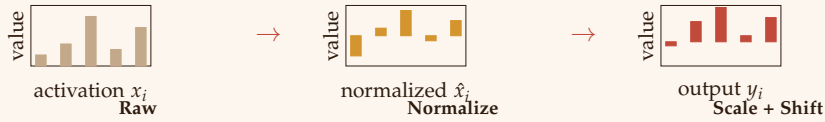


Figure 5: The BatchNorm pipeline: raw activations $\rightarrow$ zero-mean/unit-variance $\rightarrow$ learnable affine transform $\gamma\hat{x} + \beta$. The learnable parameters restore whatever mean and scale the layer needs, while the normalization itself keeps the distribution stable.

The learnable γ and β are crucial: if the optimal behavior were to leave the activations untouched, BN can learn $\gamma = \sqrt{\sigma^2 + \epsilon}$ and $\beta = \mu$ to recover the identity, so the layer is never forced to normalize away useful signal.

Listing 2: BatchNorm implementation in `mini_nn/nn_utils.py`.

```

1 import numpy as np
2
3 class BatchNorm:
4     def __init__(self, dim, momentum=0.9, eps=1e-5):
5         self.gamma = np.ones(dim)      # learnable scale
6         self.beta = np.zeros(dim)      # learnable shift
7         self.momentum = momentum
8         self.eps = eps
9         self.running_mean = np.zeros(dim) # EMA, used at
            inference
10        self.running_var = np.ones(dim)
11
12    def __call__(self, x, training=True):
13        if not training:
14            # Inference: use the population statistics (EMA)
15            x_hat = (x - self.running_mean) / np.sqrt(self.
                running_var + self.eps)
16            return self.gamma * x_hat + self.beta
17
18        # Training: use the current mini-batch statistics
19        batch_mean = x.mean(axis=0)
20        batch_var = x.var(axis=0)
21        x_hat = (x - batch_mean) / np.sqrt(batch_var + self.eps)
22
23        # Update the EMA of the population statistics
24        self.running_mean = self.momentum * self.running_mean +
            (1 - self.momentum) * batch_mean
25        self.running_var = self.momentum * self.running_var +
            (1 - self.momentum) * batch_var
26
27        return self.gamma * x_hat + self.beta

```

Remark 0.1 (Training vs. Inference in BN) During training, BN uses the batch statistics. During inference (evaluation), we cannot rely

on a single batch's statistics. Instead, PyTorch maintains an exponential moving average (EMA) of the mean and variance across all training batches, and uses these population estimates during testing. This is why calling `model.eval()` is critical in PyTorch.

Beyond stabilizing training, BN acts as a mild regularizer. The noise injected by the batch statistics (since the mean/variance are slightly different for each mini-batch) slightly perturbs the network, similar to Dropout, improving generalization.

Early Stopping

The simplest, yet highly effective, form of regularization. We monitor the validation loss during training. As long as the validation loss decreases, training continues. Once the validation loss plateaus or starts to rise (indicating the model has begun to memorize noise), training is halted.

Listing 3: Pseudo-code for Early Stopping.

```

1 best_val_loss = float('inf')
2 patience = 5
3 epochs_no_improve = 0
4
5 for epoch in range(max_epochs):
6     train_loss = train_one_epoch(model)
7     val_loss = evaluate(model, val_data)
8
9     if val_loss < best_val_loss:
10         best_val_loss = val_loss
11         save_model(model) # Save the best weights
12         epochs_no_improve = 0
13     else:
14         epochs_no_improve += 1
15         if epochs_no_improve >= patience:
16             print("Early stopping triggered.")
17             break
18
19 load_best_model(model)

```

Early stopping effectively limits the optimization algorithm's capacity to overfit. By stopping early, we constrain the trajectory of the optimizer in the weight space, landing in a wider, flatter minimum (which generalizes better) rather than a sharp, deep minimum (which overfits).

The behavior this exploits is visible in almost every training run: the training loss keeps decreasing monotonically, while the validation loss eventually turns around and climbs. Early stopping identifies that turning point.

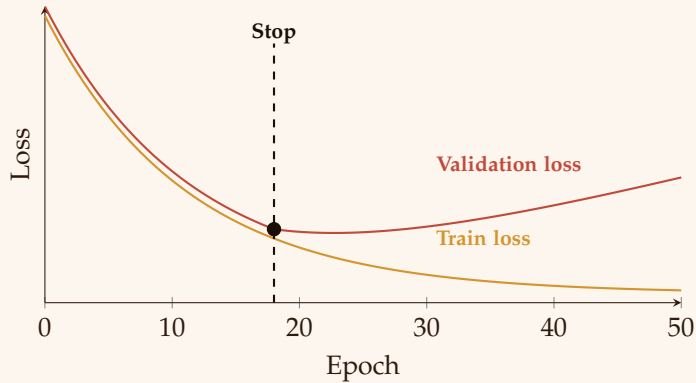


Figure 6: Training vs. validation loss. The training loss (gold) falls steadily, but the validation loss (terracotta) reaches a minimum around epoch 18 and then rises as the model starts to memorize noise. Early stopping halts training at that minimum, before overfitting begins.

A Summary of Regularization Techniques

Technique	Mechanism	Training effect	Inference effect
L2 / Weight Decay	penalty $\frac{\lambda}{2} \ \mathbf{w}\ ^2$ in the loss	weights shrink toward zero	unchanged
Dropout	random neurons zeroed per step	forces redundant representations	no drops; scaled activations
BatchNorm	normalize + scale/shift per batch	stable distributions; mild noise	uses population EMA
Early Stopping	halt when validation loss rises	shorter training, flatter minima	unchanged

Table 1: Summary of the four regularization techniques covered in this chapter. They act on different components—the loss, the architecture, the activations, or the training schedule—but all serve the same goal: reduce generalization error.

Exercises

1. Using the decomposition $\mathbb{E}[(y - \hat{y})^2] = \text{Bias}^2 + \text{Variance} + \sigma^2$, explain in one sentence each why a low-capacity and a high-capacity model generalize poorly.
2. A weight $w = 4.0$ is trained with weight decay $\lambda = 0.1$ and learning rate $\eta = 0.01$. If the data gradient is zero, write down the first three weight values.
3. Dropout scales activations by $1/(1 - p)$ during training. Why does this make inference (no scaling) consistent? What would happen to the output magnitude if the scaling were omitted?
4. BatchNorm maintains an EMA of batch statistics. Why can't the final batch's statistics be used directly at inference time?
5. In the double descent curve, what happens to the training error after the interpolation threshold? Why does the test error keep decreasing in that regime?

6. Early stopping requires a separate validation set. How would you detect overfitting if you only had access to the training set? What is the risk of doing so?
7. For each of the four regularization techniques, state which component of the learning process (loss, architecture, activations, or schedule) it modifies.

Remark 0.2 (The Transition to Architecture) *We now know how to build models (Chapter 1), how to train them (Chapter 2), and how to prevent them from memorizing (Chapter 3). However, applying a generic MLP to images results in an explosion of parameters and the loss of spatial structure. In Chapter 4, we will introduce Inductive Bias through Convolutional Neural Networks (CNNs), tailoring our architectures to the structure of the data.*